

1.1 Introduction

One of the best ways to build motivation for learning DSP concepts is to experiment with real audio signals. In this lab, we will read audio files and examine several important signal parameters. We will also explore both the time-domain and frequency-domain representations of the audio signal.

1.2 Planning of the project

We will add new code to the existing dsp26 project. Specifically, we will:

- Add code to `app_cli.py` so that we can execute the new lab functionality.
- Add a new folder, `src/dsp26/lab1_audio_sig`, to host the core code for this lab.

Before proceeding, please ensure that Lab 0 works correctly. If you encounter any issues, please reach out to the TA or the instructor for assistance.

1.3 Lab Task: Project Management (30 pts)

The project management task for Lab 1 is to:

- Add the provided code to the project.
- Add the required dependencies using `uv add <dependency_list>`.
- Run the Lab 1 command of the application, as described below.

Total points for this task: 30 pts

1.3.1 Adding code to `app_cli.py`

In the code section designated for Labs (see comments in the file), add the following code to enable the command:

```
1  dsp26 play-plot-audio --path-name <a_wave_file.wav>
```

This command can be executed from a terminal in any directory containing the specified WAV file.

`app_cli.py`:

```
1  # Lab1-audio-sig
2  @app.command()
3  def play_plot_audio(
4      path_name: str = typer.Option(
5          "",
6          "--path-name",
7          help="Optional output path name for loading an audio file. " \
8              "If set, the specified audio file will be loaded.",
9      ),
10 ):
11     """Play and plot audio signal.
12     Use --path-name to load audio file from."""
13     play_plot_audio_(path_name=path_name)
```

1.3.2 Adding Lab code under `src/dsp26/lab1_audio_sig`

The lab includes three files in this folder:

`audio_player.py`

This file contains a function that plays an audio file (in WAV format) through a GUI interface. The code was generated with the assistance of Kilo Code in VS Code. No modifications are needed—simply include it as provided.

```
1  """Simple Audio Player using simpleaudio and tkinter"""
2
3  import os
4  import tempfile
5  import time
6  import wave
7  from pathlib import Path
8  import tkinter as tk
9
10 import simpleaudio as sa
11
12 class AudioPlayer:
13     _instance = None
14     _play_obj = None
15
16     def __new__(cls):
17         if cls._instance is None:
18             cls._instance = super().__new__(cls)
19             cls._instance._initialized = False
20             return cls._instance
21
22     def __init__(self):
23         if self._initialized:
24             return
25         self._initialized = True
26         self._root = None
27         self._is_playing = False
28         self._is_paused = False
29         self._audio_file = None
30         self._play_obj = None
31         self._duration = 0
32         self._pause_offset = 0.0
33         self._play_started_at = 0.0
34         self._current_file_path = None
35         self._temp_wav_path = None
36         self._play_button = None
37         self._has_played_once = False
38
39     def _get_duration(self, file_path):
40         """Get audio duration from WAV file"""
41         try:
42             with wave.open(str(file_path), 'rb') as wav:
43                 frames = wav.getnframes()
44                 rate = wav.getframerate()
45                 duration = frames / float(rate)
46                 return duration
47         except:
```

```

48         return 30 # Default fallback
49
50     def _cleanup_temp_file(self):
51         if self._temp_wav_path and Path(self._temp_wav_path).exists():
52             try:
53                 Path(self._temp_wav_path).unlink()
54             except OSError:
55                 pass
56         self._temp_wav_path = None
57
58     def _play_from_offset(self, offset_seconds: float = 0.0):
59         """Start playback from a specific offset in seconds."""
60         if not self._current_file_path or not self._current_file_path.exists():
61             return
62
63         self._cleanup_temp_file()
64
65         offset_seconds = max(0.0, float(offset_seconds))
66         with wave.open(str(self._current_file_path), "rb") as wav_file:
67             sample_rate = wav_file.getframerate()
68             channels = wav_file.getnchannels()
69             sampwidth = wav_file.getsampwidth()
70             total_frames = wav_file.getnframes()
71             start_frame = min(int(offset_seconds * sample_rate), total_frames)
72             wav_file.setpos(start_frame)
73             remaining = wav_file.readframes(total_frames - start_frame)
74
75         with tempfile.NamedTemporaryFile(suffix=".wav", delete=False) as tmp:
76             self._temp_wav_path = tmp.name
77
78         with wave.open(self._temp_wav_path, "wb") as out_file:
79             out_file.setnchannels(channels)
80             out_file.setsampwidth(sampwidth)
81             out_file.setframerate(sample_rate)
82             out_file.writeframes(remaining)
83
84         self._play_obj = sa.WaveObject.from_wave_file(
85             self._temp_wav_path).play()
86         self._is_playing = True
87         self._is_paused = False
88         self._play_started_at = time.time() - offset_seconds
89
90     def _play(self):
91         """Play audio or replay from the beginning after the first run."""
92         try:
93             if not self._audio_file:
94                 return
95
96             if self._has_played_once:
97                 if self._play_obj:
98                     self._play_obj.stop()
99                 self._cleanup_temp_file()
100                 self._pause_offset = 0.0
101                 self._is_playing = False
102                 self._is_paused = False
103                 self._play_from_offset(0.0)
104             if self._play_button:
105                 self._play_button.config(text="Replay")

```

```

106         return
107
108         if not self._is_playing:
109             self._pause_offset = 0.0
110             self._play_from_offset(0.0)
111             self._has_played_once = True
112             if self._play_button:
113                 self._play_button.config(text="Replay")
114     except Exception as e:
115         print(f"Error playing: {e}")
116
117     def _pause(self):
118         """Pause audio by stopping current playback and storing offset."""
119         if self._is_playing and self._play_obj:
120             self._pause_offset = max(0.0, time.time() - self._play_started_at)
121             self._play_obj.stop()
122             self._is_playing = False
123             self._is_paused = True
124
125     def _resume(self):
126         """Resume audio from the last paused offset."""
127         if self._is_paused and self._current_file_path:
128             self._play_from_offset(self._pause_offset)
129
130     def _replay(self):
131         """Replay audio from start and reset pause state."""
132         if self._play_obj:
133             self._play_obj.stop()
134         self._cleanup_temp_file()
135         self._pause_offset = 0.0
136         self._is_playing = False
137         self._is_paused = False
138         if self._current_file_path:
139             self._play_from_offset(0.0)
140
141     def _exit(self):
142         """Exit the player."""
143         self._is_playing = False
144         self._is_paused = False
145         self._pause_offset = 0.0
146         if self._play_obj:
147             self._play_obj.stop()
148         self._cleanup_temp_file()
149         if self._root:
150             self._root.destroy()
151         self._instance = None
152
153     def play(self, file_path):
154         """Main method to play audio file"""
155         if isinstance(file_path, str):
156             file_path = Path(file_path)
157
158         if not file_path.exists():
159             raise FileNotFoundError(f"File not found: {file_path}")
160
161         # Close any existing instance
162         if self._root:
163             self._exit()

```

```
164         time.sleep(0.1)
165
166     # Load audio file
167     print(f"Loading: {file_path}")
168     self._current_file_path = file_path
169     self._audio_file = sa.WaveObject.from_wave_file(str(file_path))
170     self._duration = self._get_duration(file_path)
171     print(f"Duration: {self._duration:.2f} seconds")
172
173     # Create GUI
174     self._root = tk.Tk()
175     self._root.title("Audio Player")
176     self._root.geometry("500x300")
177
178     # Label
179     label = tk.Label(
180         self._root,
181         text=f"Playing: {file_path.name}",
182         font=("Arial", 12)
183     )
184     label.pack(pady=10)
185
186     # Duration label
187     duration_label = tk.Label(
188         self._root,
189         text=f"Duration: {self._duration:.2f} seconds",
190         font=("Arial", 10)
191     )
192     duration_label.pack(pady=5)
193
194     # Button frame
195     button_frame = tk.Frame(self._root)
196     button_frame.pack(pady=10)
197
198     # Buttons
199     buttons = [
200         ("Play", self._play),
201         ("Pause", self._pause),
202         ("Resume", self._resume),
203         ("Exit", self._exit),
204     ]
205
206     self._play_button = None
207     row = 0
208     col = 0
209     for text, cmd in buttons:
210         btn = tk.Button(
211             button_frame,
212             text=text,
213             width=10,
214             height=2,
215             command=cmd
216         )
217         if text == "Play":
218             self._play_button = btn
219         btn.grid(row=row, column=col, padx=5, pady=5)
220         col += 1
221         if col > 2:
```

```

222         col = 1
223         row += 1
224
225     # Window close handler
226     self._root.protocol("WM_DELETE_WINDOW", self._exit)
227
228     # Start main loop
229     self._root.mainloop()
230
231
232 # -----
233 # Public API
234 # -----
235
236 def play_audio(file_path):
237     """Play an audio file with GUI."""
238     os.environ['PYTHON_GIL'] = '0'
239     player = AudioPlayer()
240     player.play(file_path)

```

audio_sig.py

This file contains several utility functions for audio signal analysis, including:

- Retrieving and displaying audio file metadata (e.g., sample rate, number of channels, bit depth, and duration).
- Plotting the time-domain waveform of the audio signal.
- Visualizing the frequency-domain spectrum using various representations (e.g., magnitude spectrum, power spectral density, and spectrogram).

These functions form the core analytical components of the lab and are called by the main CLI command.

```

1  import wave
2
3  import librosa.display
4  import matplotlib.pyplot as plt
5  import numpy as np
6
7  ## Print various parameters of an audio file
8  # Open the WAV file in read mode to access the properties
9  def print_wave_file_properties(wav_file_path):
10     wav_file_path = str(wav_file_path)
11     with wave.open(wav_file_path, 'r') as wav_file:
12         print(f"\nDetailed properties of {wav_file_path} -----")
13         # Number of audio channels
14         n_channels = wav_file.getnchannels()
15         print("Number of channels:", n_channels)
16
17         # Sample width in bytes
18         sample_width = wav_file.getsampwidth()
19         print("Sample width (bytes):", sample_width)
20
21         # Frame rate (sample rate)
22         frame_rate = wav_file.getframerate()
23         print("Frame rate (sample rate):", frame_rate)
24

```

```

25         # Number of audio frames
26         n_frames = wav_file.getnframes()
27         print("Number of frames:", n_frames)
28
29         # Duration of the audio in seconds
30         duration = n_frames / float(frame_rate)
31         print("Duration (seconds):", duration)
32         # Other properties can be accessed similarly
33
34
35     # Display the waveform from start_s to end_s seconds
36     def plot_audio_sig_in_td(audio, sample_rate, start_s=0.0, end_s=None):
37         start_idx = int(start_s * sample_rate)
38         if end_s == None:
39             end_idx = len(audio)
40             end_s = end_idx / sample_rate
41         else:
42             end_idx = int(end_s * sample_rate)
43
44         segment = audio[start_idx:end_idx]
45         time_length = len(segment) / sample_rate
46
47         plt.figure(figsize=(16, 9))
48         librosa.display.waveshow(segment, sr=sample_rate, color="blue")
49
50         ax = plt.gca()
51         ax.set_xlim(0, time_length)           # keep the selected window visible
52         ax.set_xticks([0, time_length])       # left and right edge only
53         ax.set_xticklabels([f"{start_s}", f"{end_s}"]) # label as start_s/end_s
54
55         plt.title(f"Waveform from {start_s:.1f}s to {end_s:.1f}s")
56         plt.xlabel("Time (s)")
57         plt.ylabel("Amplitude")
58         plt.show()
59
60
61     def plot_audio_sig_in_fd(audio, sample_rate, fd_format='linear'):
62         spec = librosa.stft(audio, n_fft=512, hop_length=160, win_length=400)
63         spec_db = librosa.amplitude_to_db(np.abs(spec), ref=np.max)
64
65         plt.figure(figsize=(16, 9))
66         librosa.display.specshow(spec_db, y_axis=fd_format, x_axis='time',
67                                   sr=sample_rate, hop_length=160)
68         plt.colorbar()
69         plt.title('Spectrogram of an Audio Signal')
70         plt.show()

```

recipes.py

This file serves as the bridge between the CLI commands defined in `app_cli.py` and the core functionality implemented in the other module files. It contains the wrapper functions that:

- Parse and validate inputs received from the command line.
- Call the appropriate core functions from `audio_sig.py` and `audio_player.py`.
- Handle any necessary data transformation or error handling.

This separation of concerns keeps the CLI layer clean and focused on command parsing, while the core logic remains modular and reusable.

`recipes.py`:

```
1  from pathlib import Path
2
3  import librosa
4
5  from dsp26.lab1_audio_sig.audio_player import play_audio
6  from dsp26.lab1_audio_sig.audio_sig import (
7      print_wave_file_properties,
8      plot_audio_sig_in_td,
9      plot_audio_sig_in_fd,
10 )
11
12
13 def play_plot_audio(path_name: str = ""):
14     if path_name:
15         audio_file_path = path_name
16     else:
17         # Get the directory containing this Python file:
18         # dsp26/src/dsp26/lab1_audio_sig/recipes.py
19         current_dir = Path(__file__).resolve().parent
20         # Go up:
21         # lab1_audio_sig -> dsp26 -> src -> project root (dsp26)
22         project_root = current_dir.parents[2]
23         # Construct the audio file path
24         audio_file_path = project_root / "audio_files" / "2086-149220-0033.wav"
25
26     play_audio(audio_file_path)
27
28     print_wave_file_properties(audio_file_path)
29
30     audio, sample_rate = librosa.load(audio_file_path, sr=None)
31
32     plot_audio_sig_in_td(audio, sample_rate)
33     plot_audio_sig_in_td(audio, sample_rate, 0.4, 1.4)
34
35     plot_audio_sig_in_fd(audio, sample_rate)
36     plot_audio_sig_in_fd(audio, sample_rate, 'log')
37     plot_audio_sig_in_fd(audio, sample_rate, 'mel')
```

1.3.3 Adding dependencies

You may need to add new dependencies for this lab and future labs. We will not explicitly specify which dependencies are required. A common approach is to simply run the application; if a package named is missing, you can add it using:

```
1  uv add <a_package>
```

Alternatively, you can proactively add any packages you anticipate needing.

1.3.4 Running the provided code

This process may take a few steps if dependencies are not installed correctly. You may need to run the following commands iteratively until all dependencies are satisfied:

```
1 uv add <a_package>
2 dsp26 play-plot-audio
```

Note:

- Once installed via uv, the app can be run from anywhere on your system.
- If new dependencies are added, you may need to perform a **forced local installation**. Refer to the uv cheatsheet for the specific command if this becomes necessary.

1.3.5 Artifacts for the project management task

To demonstrate that your code works correctly, take screenshots of the following:

- The output of running `dsp26--help`.
- The Audio Player window when running `dsp26play-plot-audio`.
- Each of the **5 figures** displayed after closing the Audio Player window. (Note: You must close one figure to reveal the next.)

1.4 Experimental task (20 pts)

Once the basic setup is complete, record a **short music audio file** with the following specifications:

- Channels: Single channel (mono)
- Sample rate: 16 kHz
- Duration: Approximately 10 seconds

You may use any music source (e.g., YouTube). Save the file in **WAV format** in a folder of your choice. Then, run the app in that folder with the specified file name to:

- Listen to the music.
- View the waveform.
- Examine the spectrum displays.

1.4.1 Artifacts for the experimental task

Take screenshots of the following:

- The second (zoomed-in) waveform figure.
- The last spectrum figure.

1.5 Programming tasks (40 pts)

Total points for this task: 40 pts

The `dsp26play-plot-audio` command currently supports one option: `--path-name`, which we have already used.

In Figure 2, we displayed a zoomed-in waveform from 0.4 to 1.4 seconds. Now, we extend this functionality to support arbitrary start and end times. The good news is that the `plot_audio_sig_in_td` function already supports this feature. Therefore, we only need to:

- Add two new options to the `dsp26play-plot-audio` command.
- Use the same default values as those defined in the `plot_audio_sig_in_td` function.

AI-Assisted Coding Note:

Without AI assistance, this coding task can be challenging due to the need to navigate multiple files and function signatures. With AI-assisted coding (e.g., using Kilo Code in VS Code), this becomes straightforward:

- Simply describe the problem and requirements to the AI, and it will generate the necessary code.
- You must still be able to read, understand, and integrate the code into the appropriate locations.

Kilo Code provides free LLM access for coding within VS Code, with no need to purchase tokens. While there is a token limit, it is more than sufficient for basic coding tasks.

1.5.1 Artifacts for the programming task

Submit the following:

- Code Snippet: The code added to `app_cli.py` and/or `lab1_audio_sig/recipes.py` for the new start and end time parameters.
- Code Explanation: A concise summary of how the command-line options work. (This can be generated by an LLM, but you should condense it to the most relevant points.)
- Screenshot of zoomed-in figure (of your music file) from 0.5 to 1.5 seconds.

1.6 Submission (10 pts)

Total points for submission: 10 pts

Submit a single PDF file containing all the artifacts collected from the tasks above:

- Project Management Task artifacts
- Experimental Task artifacts
- Programming Task artifacts

Important: The project will receive a grade only after you demonstrate the running results to the TA, in addition to submitting the collection of artifacts.